@RepeatedTest:

@RepeatedTest:

It is used to run the same test (with same input) multiple times.

Syntax:

```
@RepeatedTest(value = 5)    // run this test 5 times
void repeatedTest() {
    System.out.println("Running test...");
}

```

Useful for:

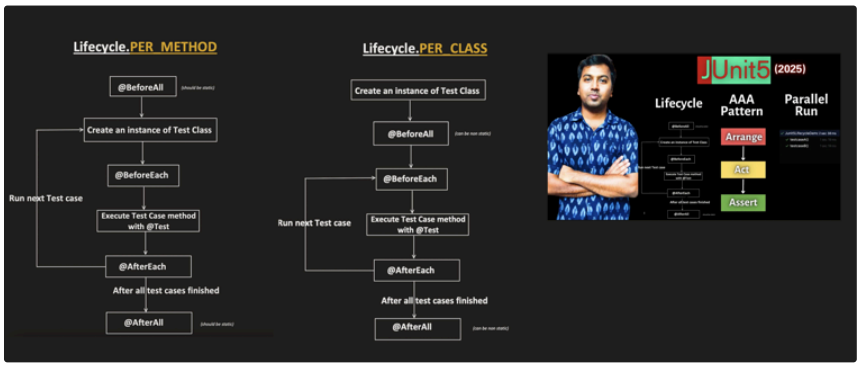
- Detecting intermittent issues.
- Testing Idempotency
- Stress testing (running code repeatedly to ensure it behaves properly under heavy and prolonged load)

Etc.

@RepeatedTest lifecycle:

We already covered the lifecycle of Test cases. And current understanding is:

- **Lifecycle.PER_METHOD** scenarios, for each Test Case new instance is created.
- **Lifecycle.PER_CLASS** scenarios, instance is created only once for all the tests.



In case of @RepeatedTest (say test need to run for 5 times), then consider each 5 Iteration as a separate/independent test run only

```
public class MyServiceTest {

    public MyServiceTest() {
        System.out.println("instance created");
    }

    @BeforeAll
    public static void beforeAllMethod() {
```

Concept && Coding

```java
        System.out.println("inside before all method");
    }

    @BeforeEach
    public void beforeEachMethod() {
        System.out.println("inside before each method");
    }

    @Test
    void testMethod() {
        System.out.println("inside testMethod");
    }

    @RepeatedTest(value = 2)
    void testRepeatedMethod() {
        System.out.println("inside testRepeatedMethod");
    }

    @AfterEach
    public void afterEachMethod() {
        System.out.println("inside after each method");
    }

    @AfterAll
    public static void afterAllMethod() {
        System.out.println("inside after all method");
    }
}
```

Exact Execution Order (PER_METHOD)

Before anything runs

- A **single** call to `@BeforeAll` (static)

Test 1 → `testMethod()`

1. **instance created**
2. **inside before each method**
3. **inside testMethod**
4. **inside after each method**

Test 2 → `testRepeatedMethod()` (1st repetition)

Each repetition = **new instance**

1. **instance created**
2. **inside before each method**
3. **inside testRepeatedMethod**
4. **inside after each method**

Test 3 → `testRepeatedMethod()` (2nd repetition)

Again new instance

Concept && Coding

1. **instance created**
2. **inside before each method**
3. **inside testRepeatedMethod**
4. **inside after each method**

After all tests

- A **single** call to `@AfterAll` (static)

So final printed output will be:

```
inside before all method

instance created
inside before each method
inside testMethod
inside after each method

instance created
inside before each method
inside testRepeatedMethod
inside after each method

instance created
inside before each method
inside testRepeatedMethod
inside after each method

inside after all method

```

RepetitionInfo Object as parameter:

- JUnit provides **RepetitionInfo** as a parameter.
- Helps to know which repetition is currently running and how many total repetitions.

```java
import org.junit.jupiter.api.RepetitionInfo;
import org.junit.jupiter.api.RepeatedTest;

public class MyServiceTest {

    @RepeatedTest(value = 2)
    void testRepeatedMethod(RepetitionInfo repetitionInfo) {

        int currentRepetition = repetitionInfo.getCurrentRepetition();
        int totalRepetitions = repetitionInfo.getTotalRepetitions();
        int failureCount = repetitionInfo.getFailureCount();
        int failureThreshold = repetitionInfo.getFailureThreshold();

        System.out.println(
                "Running Test : currentRepetition: " +
currentRepetition +
                ", totalRepetitions: " + totalRepetitions +
                ", failureCount: " + failureCount +
                ", failureThreshold: " + failureThreshold
        );
    }
}

```

Concept && Coding

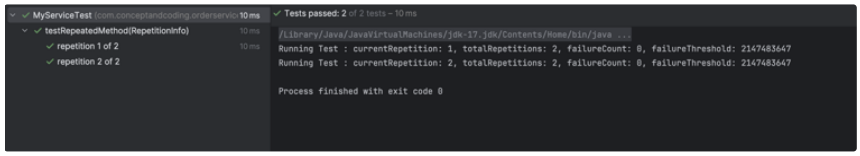
Extra Clarification (Important)

`RepetitionInfo` gives you:

Method	Meaning
`getCurrentRepetition()`	Which iteration is running (1, 2, 3...)
`getTotalRepetitions()`	Total configured repetitions
`getFailureCount()`	How many previous repetitions failed
`getFailureThreshold()`	Maximum failures allowed before aborting

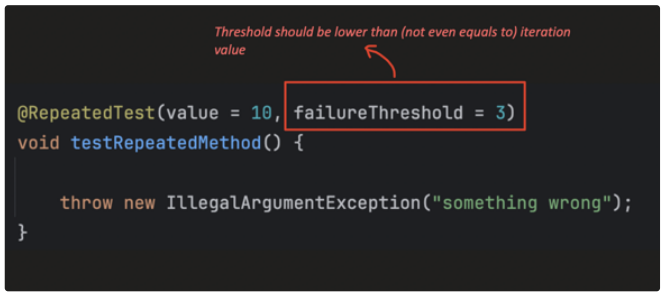
Sample Output for `value = 2`:

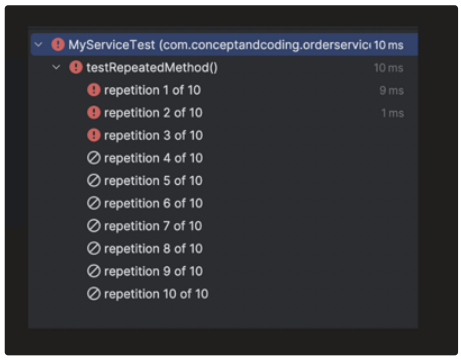
Running Test : currentRepetition: 1, totalRepetitions: 2, failureCount: 0, failureThreshold: 1
Running Test : currentRepetition: 2, totalRepetitions: 2, failureCount: 0, failureThreshold: 1



But what's this "failure Count" and "failure Threshold' ?

Failure Threshold tells how many failures are tolerated before remaining repetitions are skipped.





By default, "failureThreshold" = Integer.MAX_VALUE means all repetitions will run, even if there are failures.

Concept && Coding

```
@RepeatedTest(value = 10)
void testRepeatedMethod() {

    throw new IllegalArgumentException("something wrong");
}
```

Custom Display Name:

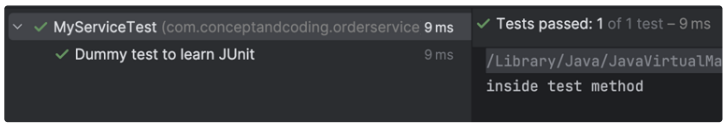
@DisplayName :

General annotation to give a **human-readable name** to a test class or method.

```
@Test
@DisplayName("Dummy test to learn JUnit")
void testMethod() {

    System.out.println("inside test method");
}

```

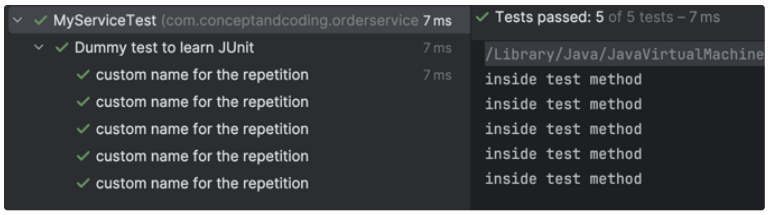


In case of @RepeatedTest, we can provide custom name too.

```
@RepeatedTest(value = 5, name = "custom name for the repetition")
@DisplayName("Dummy test to learn JUnit")
void testMethod() {

    System.out.println("inside test method");
}

```



@RepeatedTest annotation also provides fields which we can use for the name:

Concept && Coding

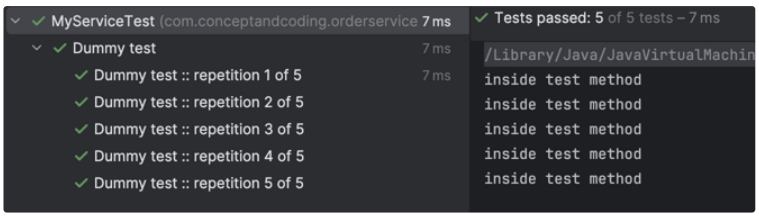
```java
@Target({ElementType.ANNOTATION_TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@API(
        status = Status.STABLE,
        since = "5.0"
)
@TestTemplate
public @interface RepeatedTest {
    String DISPLAY_NAME_PLACEHOLDER = "{displayName}";
    String CURRENT_REPETITION_PLACEHOLDER = "{currentRepetition}";
    String TOTAL_REPETITIONS_PLACEHOLDER = "{totalRepetitions}";
    String SHORT_DISPLAY_NAME = "repetition {currentRepetition} of {totalRepetitions}";
    String LONG_DISPLAY_NAME = "{displayName} :: repetition {currentRepetition} of {totalRepetitions}";
```

Example 1:

```java
@RepeatedTest(value = 5, name = RepeatedTest.LONG_DISPLAY_NAME)
@DisplayName("Dummy test")
void testMethod() {

    System.out.println("inside test method");
}

```



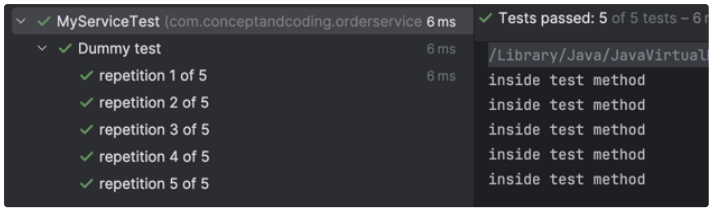
Example 2:

```java
public class MyServiceTest {

     // SHORT_DISPLAY_NAME is Default value
    @RepeatedTest(value = 5, name = RepeatedTest.SHORT_DISPLAY_NAME)
    @DisplayName("Dummy test")
    void testMethod() {

        System.out.println("inside test method");
    }
}

```



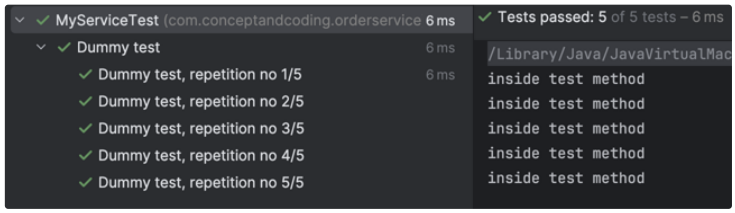
Example 3:

```java
public class MyServiceTest {

    @RepeatedTest(value = 5, name = "{displayName}, repetition no {currentRepetition}/{totalRepetitions}")
    @DisplayName("Dummy test")
    void testMethod() {
```

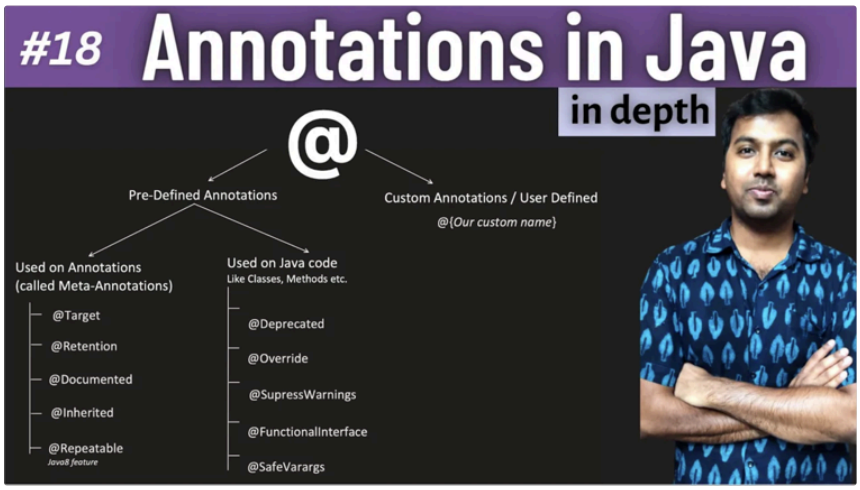
Concept && Coding

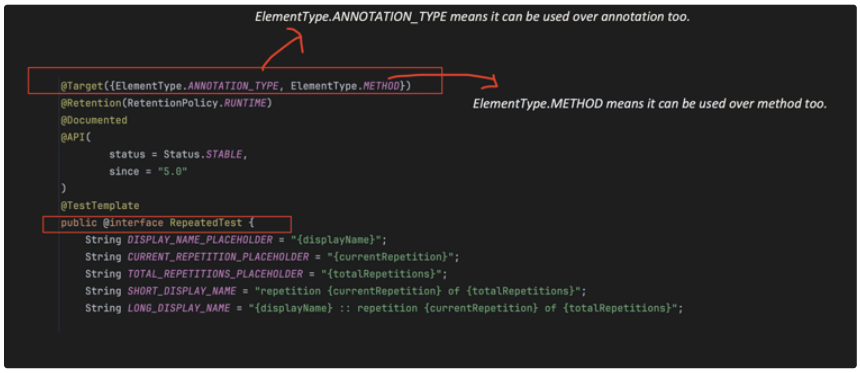
```
        System.out.println("inside test method");
    }
}
```



Using @RepeatedTest as Metadata:

In Java playlist, i have already discussed in depth that how we can create our own custom annotation, kindly check it out if there is any doubt






```
@Target({ ElementType.METHOD })
@Retention(RetentionPolicy.RUNTIME)
@RepeatedTest(10)
public @interface TenIterations {

}
```

```
@TenIterations
void testMethod() {

    System.out.println("inside test method");
}
```

Concept && Coding

```
6
```

```
MyServiceTest (com.conceptandcoding.orderservice 7 ms
    testMethod()                                  7 ms
        repetition 1 of 10                        7 ms
        repetition 2 of 10
        repetition 3 of 10
        repetition 4 of 10
        repetition 5 of 10
        repetition 6 of 10
        repetition 7 of 10
        repetition 8 of 10
        repetition 9 of 10
        repetition 10 of 10

Tests passed: 10 of 10 tests – 7 ms

/Library/Java/JavaVirtualMachines/jdk-1
inside test method
inside test method
inside test method
inside test method
inside test method
inside test method
inside test method
inside test method
inside test method
inside test method

Process finished with exit code 0
```

Concept && Coding